

Definição de classes

Bib: Java: An Introduction to Problem Solving & Programming, Walter Savitch

Cap. 5: Defining Classes and Methods

Cap. 6: More About Classes and Methods

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC

Slides inspirados nos slides da professora Manuela Sousa



Resumo

- Classes
- Métodos
- Campos
- Considerações gerais
- Encapsulamento

Resumo

- Classes
- Métodos
- Campos
- Considerações gerais
- Encapsulamento

Definição de uma classe

- Uma classe define um tipo composto
- Uma classe é constituída por *membros*
 - Campos/variáveis da classe – definem as características do tipo
 - Métodos – definem o comportamento do tipo
- Uma única classe com visibilidade pública por ficheiro
- Nome da classe igual ao nome do ficheiro com extensão `.java`
 - Convenção na definição do nome da classe: palavras com a primeira letra maiúscula
- Sintaxe:

```
public class <ClassName> {  
    <members>  
}
```

```
public class <ClassName> {  
    <fields>  
    <methods>  
}
```

Resumo

- Classes
- Métodos
- Campos
- Considerações gerais
- Encapsulamento

Método

- Definição de um método – componentes

```
<visibility> [final][static] <return_type> <name> (<parameters>)  
{  
    <body>  
}
```

Método – visibilidade

- Definição de um método

```
<visibility> [static][final] <return_type> <name> (<parameters>)  
{  
    <body>  
}
```

- Visibilidade: **public**, **private**, **protected** e **package** se omissos
 - **public**: visível em todas as classes – interface pública do tipo
 - **private**: visível apenas dentro da classe – métodos auxiliares
 - **protected**: visível nas classes derivadas e em classes do mesmo *package*
 - **package**: visível em classes do mesmo *package*

Método – exemplos com visibilidade

```
public class MethodVisibilityDemo {  
    // fields  
  
    public void doA()      { /*body*/ } // Visível em qualquer classe  
    private void doB()     { /*body*/ } // Visível apenas dentro da classe  
    protected void doC()  { /*body*/ } // Visível dentro classe, em classes derivadas e  
                                // em classes dentro do mesmo package  
    void doD()             { /*body*/ } // Visível em classes dentro do mesmo package  
  
    public static void main(String[] args) {  
        MethodVisibilityDemo demo1 = new MethodVisibilityDemo();  
        demo1.doA(); demo1.doB(); demo1.doC(); demo1.doD();  
    }  
}
```

Método – modificadores `static` e `final`

- Definição de um método

```
<visibility> [static][final] <return_type> <name> (<parameters>) {  
    <body>  
}
```
- Palavra chave `static`:
 - Com `static` -> Método da classe
 - Não precisa objeto para ser executado
 - Apenas pode alterar campos da classe
 - Sem `static` -> Método de instância
 - Precisa objeto para ser executado
 - Pode alterar campos do objeto e da classe
 - Recebe um parâmetro implícito com a referência para o próprio objeto (`this`)
- Palavra chave `final`:
 - Apenas possível em métodos de instância – impede a sobreposição (*override*) do método em classes derivadas
 - Todo o método de instância deve ser marcado como `final` por omissão, se não estiver previsto a sua sobreposição

Método – exemplos com **static** e **final**

```
public class MethodStaticAndFinalDemo {  
    // fields
```

Pode chamar quaisquer métodos da classe

Pode chamar apenas métodos estáticos da classe

```
    public void doA()          { /*body*/ } // Método de instância  
    public static void doB()   { /*body*/ } // Método da classe  
    public final void doC()    { /*body*/ } // Não pode ser sobreposto em classes derivadas  
    public static final void doD(){ /*body*/ } // Muito pouco usual; impede que o método seja  
                                     // escondido numa classe derivada
```

```
    public static void main(String[] args) {  
        MethodStaticAndFinalDemo.doB(); doD();  
        MethodStaticAndFinalDemo demo = new MethodStaticAndFinalDemo();  
        demo.doA();    demo.doC();  
        demo.doD();  
    }  
}
```

doB e **doD** não precisam de objetos para serem chamados nem do nome da classe se forem métodos desta classe

doA e **doC** precisam de um objeto para serem chamados

doB e **doD** podem ser chamados com base na referência para o objeto mas não é boa prática

Método – tipo de retorno

- Definição de um método

```
<visibility> [static][final] <return_type> <name> (<parameters>) {  
    <body>  
}
```

- Tipo de retorno do método:

- Tipo primitivo – retorna um valor – usa a instrução `return <expression>;`
 - No corpo do método pode retornar qualquer subtipo do tipo de retorno (ex: tipo de retorno long, no corpo do método pode retornar char (CONFIMAR), byte, short, int)
- Tipo classe – retorna uma referência – usa a instrução `return <obj_ref>;`
 - No corpo do método pode retornar qualquer tipo derivado
- `void` – não retorna nada – o corpo do método pode retornar com a instrução `return;`
- A instrução `return` provoca o retorno imediato do método e pode ser usada em qualquer sítio dentro do método; pode aparecer uma ou mais vezes

Método – exemplos com tipos de retorno

```
public class MethodReturnDemo {  
    // fields  
  
    public void doA()    { /*body*/ }  
    public void doB()    { return; }  
    public int doC()     { return 10; }  
    public double doD()  { return doC(); }  
}
```

Sem instrução return, retorna depois de executar o corpo do método;
permitido apenas para métodos void

Exemplo com conversão implícita

Método – nome

- Definição de um método

```
<visibility> [static][final] <return_type> <name> (<parameters>)  
{  
    <body>  
}
```

- Nome do método:

- Constituído por letras, dígitos e pelo caractere travessão (‘_’); não pode começar por dígito
- Convenção: primeira letra da palavra do método em minúscula e primeira letra das palavras seguintes em maiúscula
- Por norma são usados verbos para o nome do método

Método – exemplos com nomes

```
public class MethodNamesDemo {  
    // fields  
  
    public void _add31()          { /*body*/ }  
    public void add31()           { /*body*/ }  
    public void sumAndMultiply()  { /*body*/ }  
}
```

Método – parâmetros

- Definição de um método

```
<visibility> [static][final] <return_type> <name> (<parameters>) {  
    <body>  
}
```

- Parâmetros do método:

- Um parâmetro é declarado com o tipo e o nome
- Sem parâmetros – método sem parâmetros
- Com um ou mais parâmetros – declarações de parâmetros separadas pelo caractere ‘;’
- Na definição do método são denominados por *parâmetros formais* ou simplesmente *parâmetros*
- No contexto da chamada do método denominam-se por *parâmetros atuais* ou *argumentos*
- Na chamada há uma cópia dos argumentos para os parâmetros
 - Com tipos primitivos, é copiado o valor – alterações dos valores dos parâmetros não se refletem nos valores dos argumentos
 - Com tipos compostos, é copiada a referência – torna possível alterar o estado do objeto cuja referência é passada ao método
- Os parâmetros têm visibilidade e tempo de vida equivalente a variáveis locais do método

Método – exemplos com parâmetros

```
public class MethodParametersDemo {  
    public double doA(int x, double y, MethodReturnDemo z) {  
        x = x + 10;    z = new MethodReturnDemo();    System.out.println("z = " + z);  
        return x + y + z.doD();  
    }  
    public static void main(String[] args) {  
        MethodParametersDemo demo = new MethodParametersDemo();  
        byte b = 10;  
        MethodReturnDemo mrd = new MethodReturnDemo();  
        System.out.println("mrd = " + mrd);  
        double d = demo.doA(b, 0.5, mrd);  
        System.out.println("b = " + b + ", mrd = " + mrd + ", d = " + d);  
    } }  
}
```

```
mrd = slides.cap4.MethodReturnDemo@8efb846  
z = slides.cap4.MethodReturnDemo@2a84aee7  
b = 10, mrd = slides.cap4.MethodReturnDemo@8efb846, d = 30.5
```

Método – corpo

- Definição de um método

```
<visibility> [static][final] <return_type> <name> (<parameters>)  
{  
    <body>  
}
```

- Corpo do método:

- Conjunto de instruções envolvidas entre os caracteres chavetas ‘{’ <body> ‘}’
- O corpo de um método é um exemplo de um bloco
- Inclui a definição de variáveis locais e instruções
- Variáveis definidas dentro do método têm visibilidade apenas dentro do método e o tempo de vida corresponde à execução do método, ou seja, são variáveis de alocação automática com alocação de espaço no *stack* ou em registos do CPU

Método – exemplo com blocos

```
public class MethodBlocksDemo {  
    public static void main(String[] args) {  
        int a = 10;  
        {  
            int b = a + 20; // a é visível dentro do bloco  
        }  
        // a += b; Erro de compilação – b não é visível fora do bloco  
    }  
}
```

Método – sobrecarga (*overload*) de métodos

- Métodos da classe e métodos de instância podem ser sobrecarregados (*overloaded*)
- Os métodos têm o mesmo nome
- Os métodos distinguem-se pelos parâmetros
 - Seja em número de parâmetros
 - Seja em tipo de parâmetros
- O tipo de retorno não distingue métodos

Método – exemplos com sobrecarga de métodos

Excerto da classe java.lang.Math

Modificador and retorno	Método	Descrição
static double	<u>min</u> (double a, double b)	Returns the smaller of two double values.
static float	<u>min</u> (float a, float b)	Returns the smaller of two float values.
static int	<u>min</u> (int a, int b)	Returns the smaller of two int values.
static long	<u>min</u> (long a, long b)	Returns the smaller of two long values.

Excerto da classe java.io.PrintStream

void	<u>print</u> (boolean b)	Prints a boolean value.
void	<u>print</u> (char c)	Prints a character.
void	<u>print</u> (double d)	Prints a double-precision floating-point.
void	<u>print</u> (long l)	Prints a long integer.
void	<u>print</u> (Object obj)	Prints an object.
void	<u>print</u> (String s)	Prints a string.

Resumo

- Classes
- Métodos
- Campos
- Considerações gerais
- Encapsulamento

Campo ou variável da classe

- Campo ou variável da classe corresponde a uma característica da classe
- Podem pertencer à classe ou ao objeto
- Convencionalmente são declaradas no início da classe
- Declaração: não tem atribuído um valor inicial
`<visibility> [static] [final] <type> <name>;`
 - Na declaração não aparece o operador afetação
 - Na criação é afetada com o valor por omissão
- Definição: tem atribuído um valor inicial
`<visibility> [static] [final] <type> <name> = <expression>;`
 - Na declaração aparece o operador afetação e uma expressão que define o valor inicial
 - O campo toma o valor produzido pela expressão
 - Para tipos primitivos, a expressão deve produzir um valor do mesmo tipo ou compatível
 - Para tipos compostos, a expressão deve produzir uma referência do tipo ou subtipo

Campo – visibilidade

- Definição de um campo

`<visibility> [static][final] <type> <name>[=<expression>];`

- Visibilidade: **public**, **private**, **protected** e **package** se omissos
 - **public**: visível em todas as classes – má prática
 - **protected**: visível nas classes derivadas ou em classes do mesmo *package* – usado, quando relevante, numa hierarquia de classes com relação de herança
 - **package**: visível em todas as classes dentro do mesmo *package*
 - **private**: visível apenas dentro da classe – preserva a integridade do campo; visibilidade usada preferencialmente

Campo – modificadores **static** e **final**

- Definição de um campo
`<visibility> [static][final] <type> <name>[=<expression>];`
- Palavra chave **static**:
 - Com **static** -> Campo da classe
 - Não precisa objeto para poder ser acessado
 - Sem **static** -> Campo de instância
 - Precisa objeto para poder ser acessado
 - O valor do campo é específico do objeto
- Palavra chave **final**:
 - Pode ser usado em campos da classe ou de instância
 - O valor do campo apenas pode ser atribuído na fase de construção
 - Por omissão, todos os campos da classe devem ser marcado como **final**
- Constantes são definidas como **public, static e final**, em simultâneo
 - Por convenção, os nomes das constantes são definidos com todos os caracteres em maiúsculas e com as palavras separadas por travessão (‘_’)

Campo – tipo

- Definição de um campo

`<visibility> [static][final] <type> <name>[=<expression>];`

- Tipo do campo:

- Tipo primitivo – armazena um valor do tipo definido
- Tipo classe – armazena uma referência para um objeto do tipo especificado ou subtipo

Campo – nome

- Definição de um campo

`<visibility> [static][final] <type> <name>[=<expression>];`

- Nome do campo:

- Constituído por letras, dígitos e pelo caractere travessão (‘_’); não pode começar por dígito
- Convenção: igual ao método, ou seja, primeira letra da palavra do campo em minúscula e primeira letra das palavras seguintes em maiúscula
- Por norma, são usados substantivos para atribuir o nome do campo

Campo – exemplos

```
enum Gender {female, male};
```

```
public class FieldsDemo {  
    public static final double PI = 3.14159;    // Constante - acessível em todas as classes da  
                                                // aplicação (FieldsDemo.PI)  
    private String name;                      // Iniciada com o valor por omissão (null) e  
                                                // visível apenas dentro da classe  
    protected int age = 100;                  // Iniciada com o valor 100 e visível dentro da classe e  
                                                // classes derivadas  
    private final Gender gender = Gender.female;    // Iniciada apenas ou na declaração ou no construtor e  
                                                // é iniciada uma única vez  
    private static int oldest = 122;          // Acessível por todos os objetos desta classe  
  
    // Constructors and Methods  
}
```

Resumo

- Classes
- Métodos
- Campos
- Considerações gerais
- Encapsulamento

Construção de um objeto – métodos construtores

- Métodos especiais - usados para iniciar o estado de um objeto quando for criada uma nova instância (no contexto do operador new)
- Não podem ser chamados durante o tempo de vida dos objetos
- Na definição do construtor não é representado o tipo de retorno
- Usual a sobrecarga de construtores
 - Um construtor pode chamar outro construtor da mesma classe e tem de ser a primeira instrução (`this(<argumentos>)`)
- Na ausência da definição de qualquer construtor é fornecido o *construtor por omissão*:
 - Sem parâmetros
 - Inicia os campos de instância do objeto com valores por omissão
 - Com a definição de um qualquer construtor deixa de ser fornecido o construtor por omissão

Referência `this`

- A referência `this` representa a referência para o próprio objeto
 - Os métodos de instância e construtores recebem um parâmetro implícito com esta referência
- Pode ser usado para aceder aos campos de instância (`this.<fieldName>`) ou a outros métodos de instância da mesma classe (`this.<methodName>`), no entanto pode ser omitido (`<fieldName>` ou `<methodName>`)
- Relevante para desfazer ambiguidades quando nomes de campos da classe e nomes de parâmetros de métodos são iguais

Construtores e `this` – exemplos

```
enum Gender {female, male};
```

```
public class FieldsDemo {
```

```
    // Fields
```

```
    // Constructors
```

```
    public FieldsDemo(String name, int age, Gender gender) {  
        this.name = name;    this.age = age;    this.gender = gender;    // Pode incluir mais código  
    }
```

```
    public FieldsDemo(String name, Gender gender) {  
        this.name = name;    this.gender = gender;    // Pode incluir mais código  
    }
```

```
    public FieldsDemo(String name, int age) {  
        this(name, age, Gender.female);    // Pode incluir mais código  
    }
```

```
    public FieldsDemo() {    this("Sem nome", 0, Gender.male);    // Pode incluir mais código    }  
    // Methods
```

```
}
```

A utilização da referência `this` é obrigatória para distinguir os nomes dos campos dos nomes dos parâmetros. A referência `this` pode ser usada em qualquer método de instância

A chamada de outro construtor da classe tem de ser a primeira instrução

Construtor sem parâmetros. Deixa de ser fornecido por omissão com a definição de outros construtores. Pode ser definido explicitamente

Métodos de acesso e de modificação

- Tipicamente, uma classe implementa métodos públicos de acesso e de modificação relativos a campos privados
 - Permitem, externamente, obter dados privados de um objeto
 - Permitem, externamente, alterar dados privados de um objeto
- *Accessors methods* – tipicamente começam com `get`
 - Aplicado a campos de tipos primitivos, não constitui qualquer problema
 - Aplicado a campos do tipo classe, pode expor o problema *privacy leaks*
 - Externamente não se consegue alterar a referência para o objeto mas consegue-se alterar o estado do objeto cuja referência é retornada (assim o tipo do objeto providencie métodos para o fazer)
- *Mutators methods* – tipicamente começam com `set`
 - Devem ser escritos de forma a garantir a integridade dos dados do objeto

Variáveis

- Variáveis do tipo classe são diferentes das variáveis do tipo primitivo na forma como armazenam valores
- Todas as variáveis são implementadas reservando espaço em memória para o valor que as variáveis vão armazenar
 - Uma variável do tipo primitivo armazena um valor do tipo primitivo
 - Uma variável do tipo classe armazena um endereço em memória onde o objeto da classe está localizado

Variáveis e referência (1 de 2)

- O valor de um tipo primitivo particular requer sempre a mesma quantidade de memória
 - Ex: a variável do tipo `int` requer sempre 4 bytes
- Um objeto do tipo classe pode ser arbitrariamente grande
 - Ex: um objeto do tipo `String` pode estar vazio, ou pode conter 1, 120, 5280, ou mais caracteres
- Contudo, existe sempre um limite de dimensão para o endereço
- O endereço em memória onde o objeto é armazenado é denominado referência para o objeto

Variáveis e referência (2 de 2)

- Variáveis do tipo classe comportam-se de forma diferente de variáveis do tipo primitivo
- Uma só variável do tipo primitivo está associada com uma posição em memória onde o valor da variável está armazenada
- Múltiplas variáveis do tipo classe podem conter a mesma localização em memória onde o objeto está armazenado (mesma referência). O objeto pode ser acessado e modificado usando qualquer uma das variáveis
 - Ex: `Xpto x1, x2 = new Xpto(); x1 = x2;`
- Variáveis do tipo classe que referenciam o mesmo objeto são denominadas *aliases*

Operador == com variáveis do tipo classe

- A utilização do operador de comparação == com variáveis do tipo classe, testa se as variáveis são *aliases* uma da outra e não se referenciam objetos com dados iguais (mesmo estado)
- Para testar a igualdade dos dados dos objetos, deve usar-se o método `equals`
- O critério para considerar dois objetos iguais depende de classe para classe
 - Em alguns casos, dois objetos são “iguais” quando o valor de apenas uma variável de instância, dentro do conjunto de todas as variáveis de instância, é igual
 - Noutros casos, dois objetos são “iguais” quando o valor de todas as variáveis de instância são iguais
 - Pode implicar definir o método `equals` na classe cujos objetos vão ser comparados
- A definição do método `equals` deve ser sempre com a assinatura que está definido na classe `Object` (no tema herança perceber-se-á porquê)
`boolean equals(Object o)`

Referência `null`

- Uma variável do tipo classe pode ser iniciada com a referência `null`
 - O compilador pode exigir que a variável seja iniciada e, no momento da declaração da variável, pode não fazer sentido instanciar o objeto
 - Ex: `String line = null;`
 - No exemplo, a variável `line` não tem uma referência válida para objeto
- A referência `null` é convertível para qualquer valor do tipo referência
- Os operadores `==` e `!=` são usados para determinar se a variável tem o valor `null`
 - Ex: `boolean isLineNull = line == null;`
- Uma variável iniciada com `null` não pode ser usada para invocar métodos. A tentativa de a usar produz como resultado uma exceção *“null pointer exception”*
 - Ex: `String line == null; int len = line.length();`

Método toString

- Método para obter uma *string* representativa do estado do objeto
- Tal como o método `equals` é um método definido na classe `Object`

```
public class MethodNoToString {  
    private int field1 = 100; private String field2 = "Xpto";  
    public static void main(String[] args) {  
        MethodNoToString mnts = new MethodNoToString();  
        System.out.println(mnts.toString());  
        System.out.println(mnts);  
    }  
}
```

```
MethodNoToString@27716f4  
MethodNoToString@27716f4
```

```
public class MethodToString {  
    private int field1 = 100; private String field2 = "Xpto";  
    public String toString() { return field2 + ' ' + field1; }  
    public static void main(String[] args) {  
        MethodToString mts = new MethodToString();  
        System.out.println(mts.toString());  
        System.out.println(mts);  
    }  
}
```

```
Xpto 100  
Xpto 100
```

Resumo

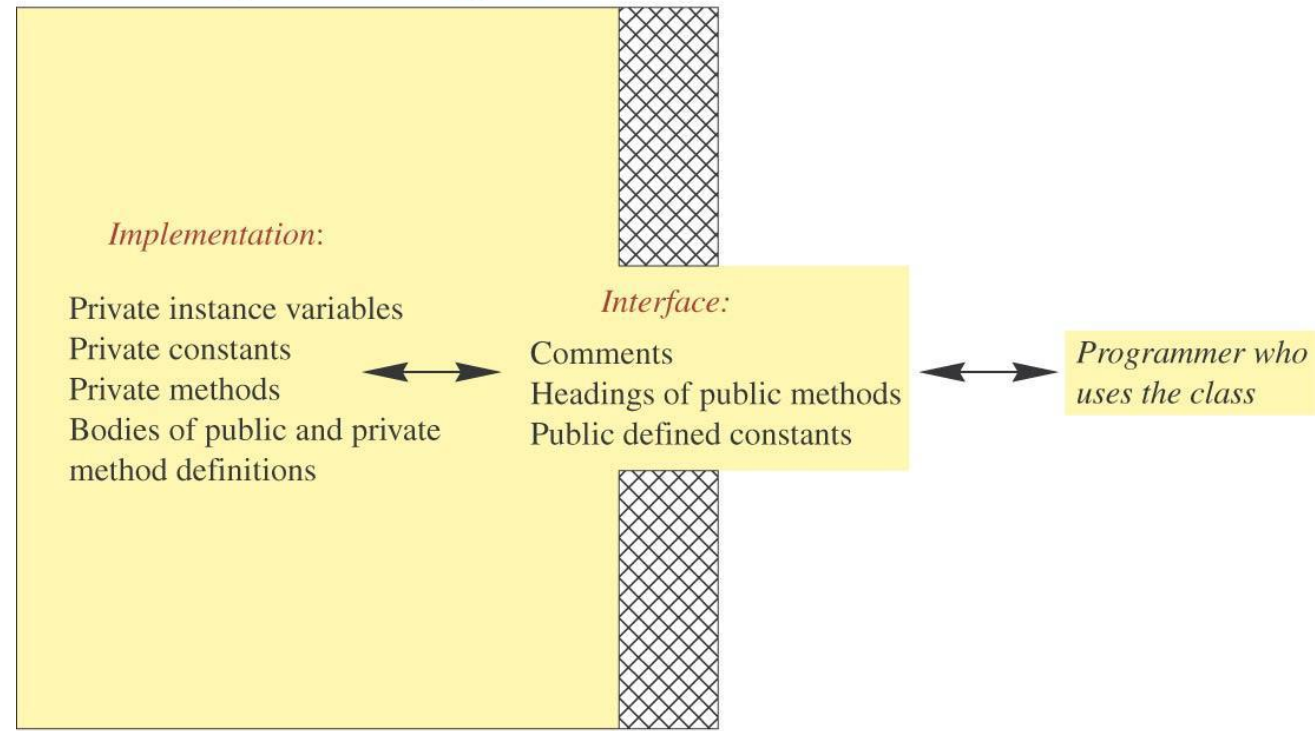
- Classes
- Métodos
- Campos
- Considerações gerais
- Encapsulamento

Encapsulamento - conceito

- Encapsulamento significa expor a informação necessária e suficiente para o programador utilizar uma classe de um determinado tipo sem que tenha de conhecer em pormenor detalhes da sua implementação
- Então, o que é que deve ser exposto ao programador?
 - Documentação da interface pública da classe (métodos com visibilidade pública)
 - Descrição do que o método faz, dos parâmetros e do retorno
 - Assinatura dos métodos públicos
 - Definições das constantes públicas
 - Tudo o resto “é a mais” (*information overflow*)

Encapsulamento

A Well-Encapsulated Class Definition



A well-encapsulated class definition has no public instance variables.

Display 4.13

Encapsulation

Encapsulamento – características

- Encapsulamento deve permitir modificação da implementação (melhorias, modificações, simplificações, etc.) sem que seja necessário modificar qualquer programa ou classe que use a classe
- Encapsulamento combina os dados e os métodos numa só entidade ocultando os detalhes da implementação
- Um *Abstract Data Type* (ADT) é basicamente o mesmo que uma definição de classe bem encapsulada.

Encapsulamento - guião

- Anteceder a definição da classe com os comentários que devem ser vistos pelos programadores que usam a classe
- Declarar como `private` todas as variáveis de instância da classe.
- Providenciar métodos públicos apropriados de acesso e modificação das variáveis de instância
- Providenciar métodos públicos que permitam ao programador usar a classe apropriadamente
- Anteceder cada método público com o comentário especificando como usar o método
- Declarar como `private` os métodos só invocados por outros métodos da classe
- Usar `/** ... */` para comentar a interface com o utilizador e `//` para comentários de implementação

Descrição de uma classe em UML

```
public class UMLDem {  
    public static final String MY_CONST = "UML Demo";  
    private final int field1;  
    protected double field2;  
  
    public UMLDem(int a, double b)    { /*body*/ }  
    public final int getField1()      { /*body*/ }  
    protected double getField2()     { /*body*/ }  
    private int auxMethod(String s, int i) { /*body*/ }  
    public static void staticMethod(char c) { /*body*/ }  
}
```

Diagrama estático de classe

